CAPSTONE PROJECT REPORT

Healthcare Knowledge Assistant

Multi-Agent Retrieval-Augmented Generation with LangGraph, FastAPI and Streamlit

A complete implementation and execution report

Prepared from the supplied capstone specification, project source, validation records, and user-provided screenshots.

Report date: 14 October 2026

Chosen domain: Healthcare information and general health education

PROJECT FACT	RECORDED RESULT
Knowledge base	100 healthcare FAQ records; machine-readable JSON and printable PDF
Core stack	Python 3.12 · LangGraph · LangChain tool · FAISS · GPT-5 mini integration
Application	FastAPI REST API and Streamlit chat UI
Local validation	8 tests passed; Ruff checks passed; live local API/UI smoke test captured
Deployment status	GitHub Actions workflow authored; no successful GitHub or Azure deployment run captured

Scope and evidence statement

Container packaging was intentionally excluded at the user's request. This report separates validated local behavior from Azure/GitHub deployment steps that were not executed or verified.

Healthcare safety: the assistant is for education only and does not diagnose, prescribe, or replace clinical care.

Executive Summary

This project implements a modular, retrieval-grounded healthcare FAQ assistant in response to the supplied “Capstone Project 2: Multi-Agent RAG Pipeline with LangGraph, FastAPI and CI/CD Deployment” specification. The selected use case is a public-facing healthcare knowledge assistant. Its knowledge base contains 100 curated, conservative FAQs with category metadata and WHO/CDC reference URLs. FAISS semantic retrieval is combined with BM25-style lexical scoring and reciprocal-rank fusion; retrieved content is handed to a LangChain RAG tool and orchestrated through a LangGraph workflow.

The workflow coordinates a planner, retriever agent, RAG tool, consultation specialist, grounding validator and bounded retrieval retry. The diagnosis-support role intentionally declines individualized diagnosis and redirects users toward qualified care. FastAPI exposes health, query and feedback routes; Streamlit provides chat, source/context disclosure and helpful/not-helpful feedback. Evaluation scripts compare a first-document baseline with agentic evidence-only responses over 20 FAQ prompts.

The recorded evaluation averages were baseline BLEU-1 0.514, ROUGE-L 0.615, relevance 0.977; agentic BLEU-1 0.272, ROUGE-L 0.411, relevance 0.977. The values come from lexical smoke tests, not clinical assessment or model-quality certification.

Important distinction

The Azure screenshots show a Foundry Project endpoint being placed in the original app’s Azure OpenAI endpoint variable, and the user reported HTTP 503. The screenshot key is fully redacted in this report. The project code was subsequently updated to use the OpenAI-compatible model inference endpoint, but the Azure connection could not be tested in this sandbox because no verified credential was available. No successful Azure deployment is claimed.

Report Contents

- 1. Project context and requirements mapping
- 2. Dataset, preparation and knowledge base
- 3. Retrieval, generation and agentic workflow
- 4. Feedback loop and evaluation
- 5. FastAPI and request/response design
- 6. Streamlit interface and agent trace
- 7. Execution record and Azure troubleshooting
- 8. CI/CD, deployment and evidence boundaries
- 9. Conclusion, limitations and next steps
- 10. References and verification appendix

1. Project Context and Requirements Mapping

1.1 Problem statement

Conventional FAQ chatbots can return unsupported or poorly sourced answers. The capstone calls for a RAG pipeline embedded within a multi-agent workflow, with retrieval, answer generation, validation, feedback, an API/UI surface and CI/CD. This implementation constrains answers to public educational FAQ evidence and exposes cited sources and retrieval context for review.

1.2 Chosen use case and safety boundary

The selected domain is healthcare knowledge assistance. The system addresses broad preventive-health and care-navigation questions using public-facing FAQ records; it does not use patient records, private notes or personal health data. The capstone's "Diagnosis Support Agent" role is adapted into a diagnosis-boundary specialist: individualized diagnosis requests are declined rather than inferred. This is a deliberate safety decision, not an accidental omission.

1.3 Requirements-to-evidence map

Capstone requirement	Implemented component	Evidence / status
Prepare domain corpus; store searchable documents	100 FAQ JSON records; FAQ PDF; metadata and source links	Count verified; FAQ PDF included
Chunk, embed and index in FAISS	Whole FAQ documents retained under 700 characters; longer text split with 100-character overlap; local deterministic hash embeddings	Index created on startup; sample retrieval returned FAQ citations
Hybrid search	FAISS similarity + BM25-style lexical rank + reciprocal-rank fusion	Implementation in src/retrieval/store.py
RAG callable as a tool; specialized agents collaborate	LangChain retrieve_health_evidence tool called by retriever agent inside LangGraph	Live API sample includes planner → retriever → tool → consultation → validator trace
Memory, validation, bounded retry	Per-session recent-turn memory; lexical grounding heuristic; retry once with larger k	Graph state and trace implemented; tests pass
Feedback loop	Helpful/unhelpful and correction payload; JSONL for later offline review	POST /feedback implemented; feedback does not auto-modify medical corpus
BLEU, ROUGE, relevance comparison	20 FAQ sample, baseline vs evidence-only agentic mode	CSV and report summary table; small smoke evaluation
FastAPI and Streamlit	/health, /query, /feedback; Swagger docs; chat UI with context/source display	Local API and UI screenshots included
CI/CD and deployment	Python 3.12 CI, Ruff, pytest, source ZIP artifact, optional Azure App Service deployment workflow	Workflow authored; GitHub execution / cloud deployment not verified

User-directed scope change

The source PDF recommends container builds. The user explicitly instructed not to use containers. Docker files and container commands were removed; local Python execution and source-ZIP deployment are used instead.

2. Dataset, Preparation and Knowledge Base

2.1 Corpus design

The source corpus contains exactly 100 manually authored, conservative question-and-answer records spanning prevention, vaccines and infection prevention, nutrition, physical activity, mental health, medicines, chronic

conditions, family/reproductive health, care navigation, and everyday/environmental health. Each item carries an FAQ identifier, category, source label and source URLs. The generated PDF version makes the corpus convenient to inspect without running the application.

Records are converted into LangChain Document objects. Text is represented as Question, Answer and Category; ID, category, source and URL metadata are retained for source linking and downstream display. WHO and CDC health-topic pages are included as public reference URLs. The URLs identify broad authoritative starting points; they do not imply that every FAQ has been individually clinically reviewed against a specific guideline.

2.2 Chunking and embedding

FAQ entries remain intact when their text is 700 characters or shorter, preserving question/answer adjacency. Oversized entries are split into 700-character windows with 100-character overlap while preserving metadata. The default embedding is deterministic local feature hashing, enabling offline/no-secret indexing. This is a lightweight engineering fallback rather than a trained semantic embedding model; an Azure embedding deployment is optional and must be provisioned separately.

2.3 Vector-store behavior

The FAISS index is built from the loaded FAQ Documents and persisted under data/faiss_index. At search time the retriever gathers a larger candidate set from FAISS and a BM25-style lexical scorer, then combines their ranks with reciprocal-rank fusion. The final context size is controlled by TOP_K (default 4). If the validator requests one retry, retrieval uses a larger k. There is no reranker in the current implementation.

System Architecture Diagram

The diagram summarizes the implemented offline/local path: FAQ documents are prepared and indexed, retrieval is exposed as a reusable tool, and LangGraph sequences specialist roles before the API returns citations and validation status.

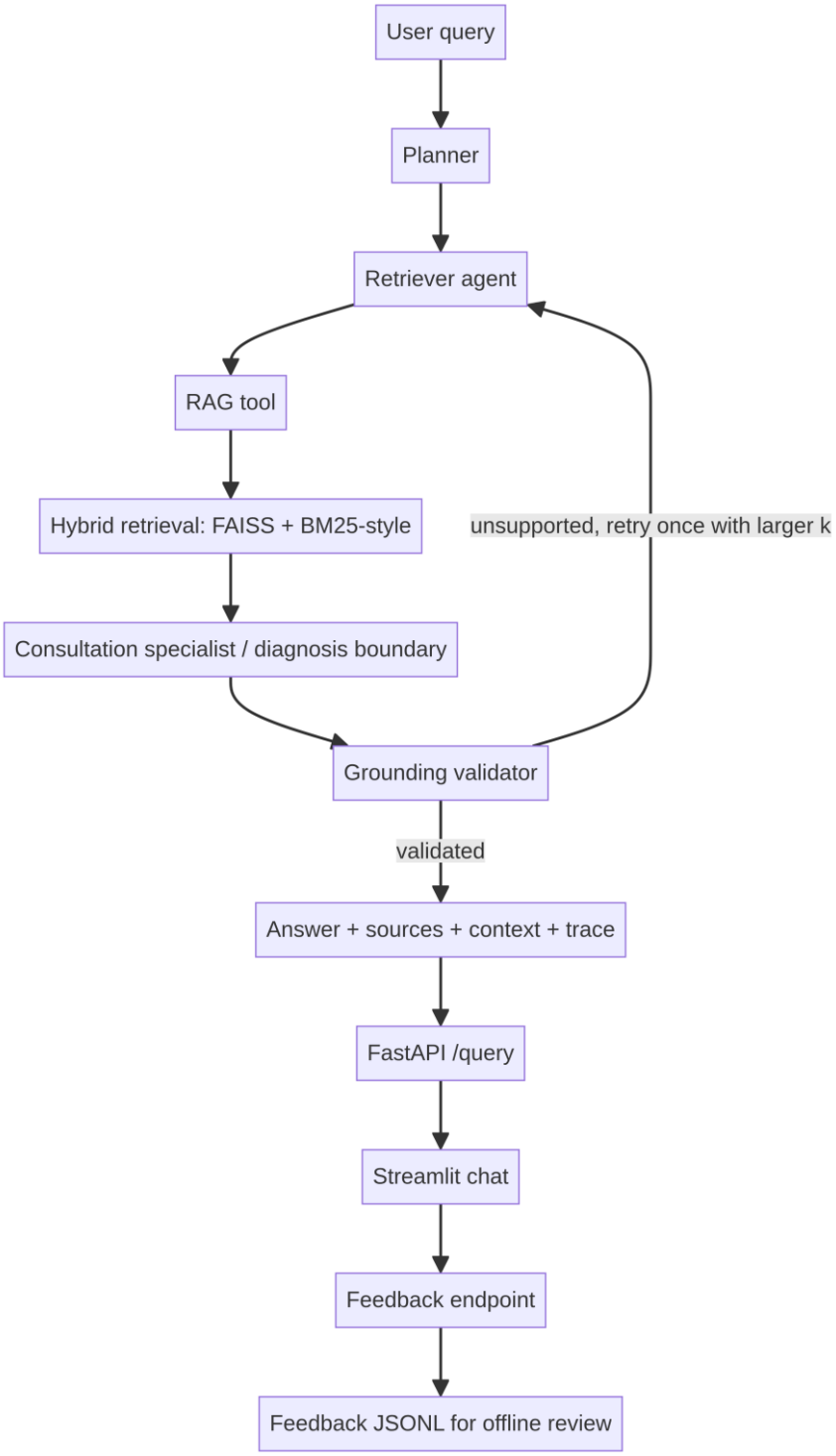


Figure 1. Rendered system workflow from the project's architecture source; this is a project diagram, not a screenshot from LangGraph Studio.

3. Retrieval and Generation Design

3.1 Request-to-answer sequence

1. Planner classifies the request at a high level as general education or a diagnosis-boundary case and records the plan in state.
2. Retriever agent invokes the reusable LangChain retrieve_health_evidence tool; the tool queries the hybrid FAISS + lexical retriever.
3. Consultation specialist receives the question and retrieved evidence. With Azure configured it calls the selected gpt-5-mini model; without Azure it returns cited excerpts from retrieved evidence.
4. Grounding validator checks whether evidence is present and computes a lexical overlap heuristic. If validation fails, a bounded retry re-runs retrieval once with a larger candidate count.
5. FastAPI returns answer text, source URLs, retrieved context, trace, heuristic confidence and validator status. Recent turns are maintained in process memory per session.

3.2 Component roles

Component	Responsibility	Key input/output
Planner	Sets coarse plan and route intent	Question → plan + trace event
Retriever Agent	Calls RAG tool with configured k	Question → serialized evidence
RAG Tool	Reusable interface to hybrid retrieval	Query → ranked FAQ evidence and IDs
Consultation Agent	Evidence-bound response; recent history	Question + context → answer
Diagnosis Boundary Agent	Declines personal diagnostic inference	Diagnosis-seeking question → safe referral
Grounding Validator	Checks evidence presence and overlap threshold	Evidence + question → verdict/confidence
Retry node	One bounded larger-k retry	Failed check → retrieval restart once

3.3 Memory and state

LangGraph state carries query, session ID, recent history, plan, evidence, answer, source IDs, trace, retry counter, validity and heuristic confidence. A session history store supplies up to four recent turns to configured Azure consultation. This memory is process-local and disappears when the service restarts; it is not an authorized clinical record store.

4. Feedback Loop and Evaluation

4.1 Feedback design

The Streamlit interface offers helpful/not-helpful controls. The FastAPI feedback endpoint accepts a session ID, query, rating from -1 to 1, and optional correction, then appends a JSON Lines record for offline review. Validator verdict metadata is separately recorded with source IDs and heuristic confidence, without logging the user's query text in that validator event. Feedback is not used to automatically rewrite the healthcare corpus, avoiding unreviewed medical-content updates.

4.2 Evaluation method

The evaluation runner uses the first 20 FAQs. For each FAQ question it obtains up to two retrieved documents, creates a baseline answer from the top document, and invokes the graph without an Azure model (evidence-only fallback). It records BLEU-1, ROUGE-L F1 and a lexical query/context relevance score in reports/evaluation.csv. Both modes share the same retrieval context when relevance is calculated; therefore equal relevance does not show that generation quality is equivalent.

Mode (n=20 each)	BLEU-1 mean	ROUGE-L F1 mean	Query/context relevance mean
Single-document baseline	0.514	0.615	0.977
Agentic evidence-only	0.272	0.411	0.977

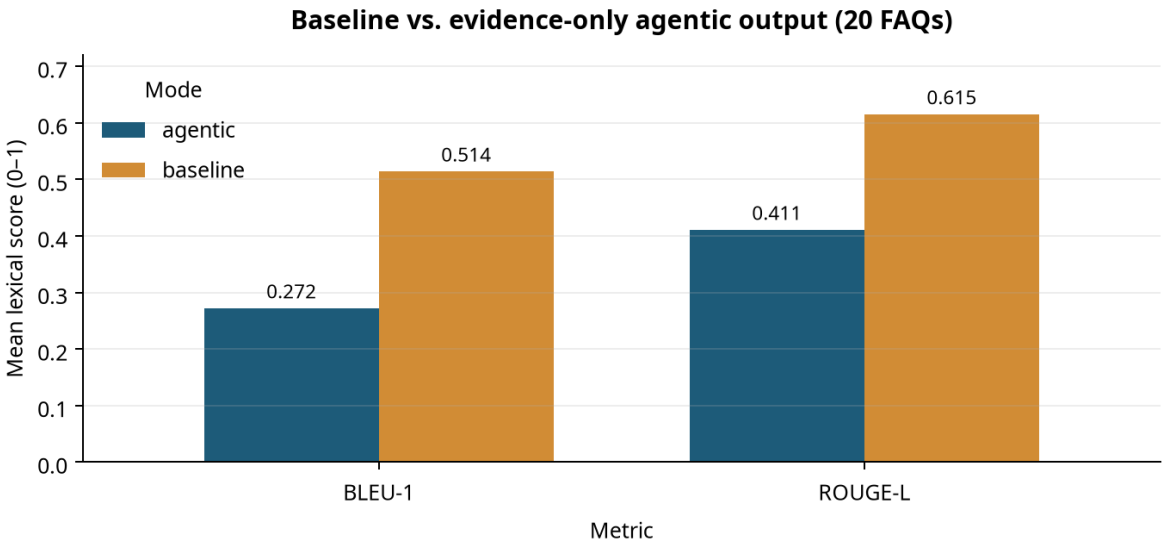


Figure 2. Data-driven chart generated from the checked-in evaluation.csv (20 questions per mode).

4.3 Interpretation and limitations

The agentic fallback produces longer evidence excerpts than the one-document baseline, so n-gram overlap with the reference FAQ answer is lower (BLEU-1 0.272 vs 0.514; ROUGE-L 0.411 vs 0.615). Relevance is 0.977 in both modes because the same retrieval context is used. These lexical measures do not evaluate clinical correctness, faithfulness, answer helpfulness or safety. The run does not use the Azure-deployed model and is not a GPT-5-mini benchmark. A clinically meaningful follow-up evaluation needs reviewed questions, blinded scoring and explicit evidence-grounded criteria.

Evaluation caution

Do not interpret heuristic confidence, overlap scores, or high lexical relevance as medical confidence or validation. Human clinical review remains necessary before any real-world clinical use.

5. FastAPI and Streamlit Application

5.1 API contract

Route	Purpose	Behavior
GET /health	Service health and configuration state	Returns status, LLM configured flag, config issue and FAQ count
POST /query	Query RAG graph	Accepts optional session_id and 2–2000 character query; returns answer, sources, context, trace and verdict
POST /feedback	Capture user evaluation	Accepts query, optional session ID, rating -1/0/1 and optional correction

5.2 Example request and response shape

```
POST /query
{ "session_id": "optional", "query": "How do vaccines help protect people?" }
```


```
{
  "session_id": "04217cde-77ae-40d3-8216-cdb978369687",
  "answer": "Vaccines train the immune system to recognize a disease-causing organism. [FAQ-011]
(excerpt)",
  "sources": ["FAQ-011: https://www.who.int/health-topics", "FAQ-011: https://www.cdc.gov/health-
topics.html"],
  "retrieved_context": ["FAQ-011: Vaccines train the immune system to recognize a disease-causing
organism."],
  "agents_invoked": ["planner", "retriever_agent", "rag_tool", "consultation_agent",
"grounding_validator"],
  "confidence": 0.99,
  "validation_passed": false
}
```


The JSON is a shortened schema example with the exact verdict/confidence from the captured local query. It returned FAQ evidence and citations, but validation_passed was false despite heuristic confidence 0.99. This discrepancy is treated as a validator-calibration issue, not proof of answer correctness. The local health response was status “ok”, 100 FAQs, and llm_configured=false because no Azure secret was configured in the sandbox.

5.3 Swagger UI evidence

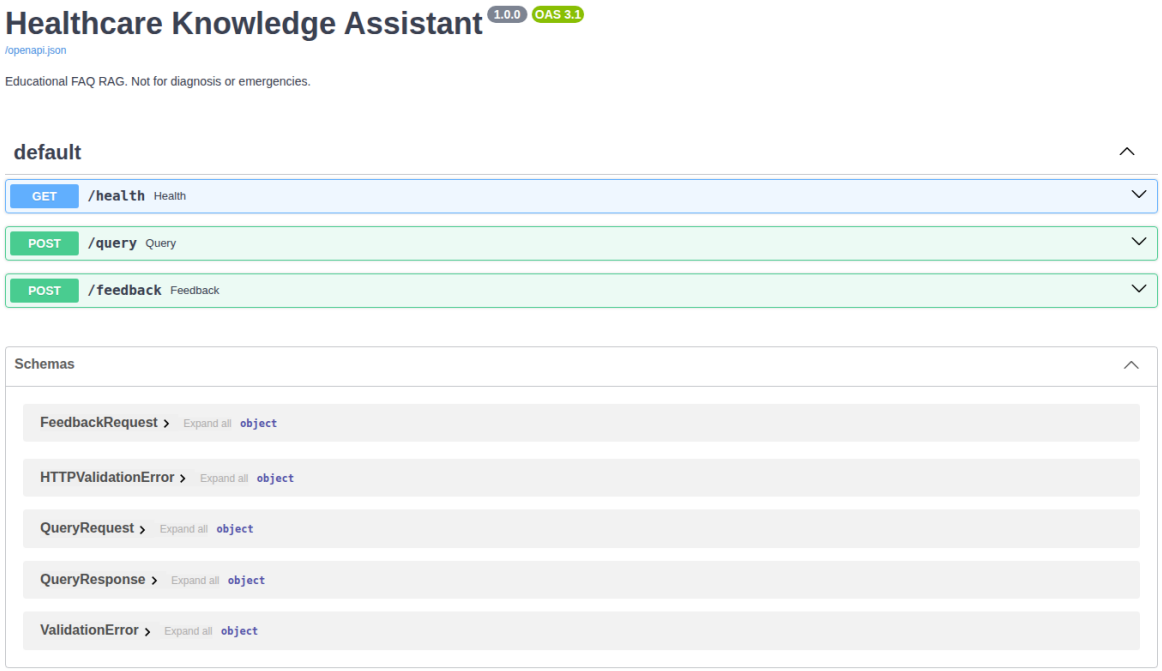


Figure 3. Swagger UI captured from the running local FastAPI server; it documents /health, /query and /feedback.

6. Streamlit Interface and Agent Trace

The Streamlit application calls the backend with requests.post, maintains a client-side message list, and displays the answer, source URLs, invoked-agent list and expandable retrieved context. It presents explicit general-education/no-diagnosis and no-sensitive-data notices. Helpful/not-helpful buttons call POST /feedback.

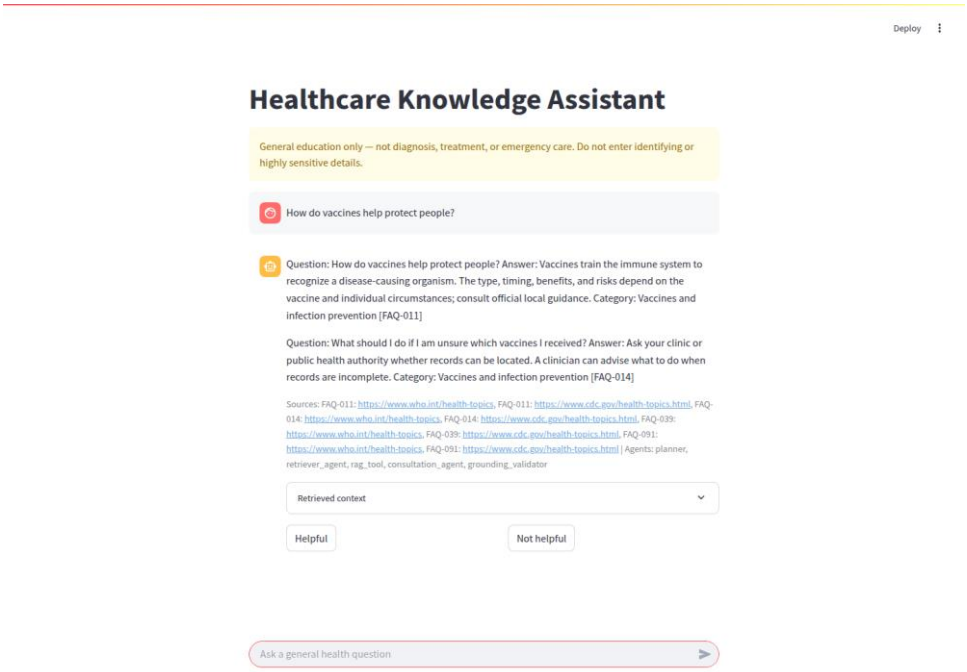


Figure 4. Actual sandbox-local Streamlit smoke-test screen after submitting "How do vaccines help protect people?"; evidence-only fallback was active.

6.1 Captured agent trace

Execution evidence	Observed result
Query	How do vaccines help protect people?
Agent trace	planner, retriever_agent, rag_tool, consultation_agent, grounding_validator
Source-linked records	FAQ-011, FAQ-014, FAQ-039, FAQ-091
Knowledge base	100 FAQs loaded; sample sources include WHO/CDC topic references
Validator outcome	validation_passed=False; confidence=0.99 (heuristic)
LLM configuration	llm_configured=False; local run used evidence-only fallback

Screenshot provenance

Figures 3 and 4 were captured from the project's actual local sandbox services during this report run. They are not screenshots from the user's Windows machine or from Azure-hosted deployment.

7. Execution Record and Validation

7.1 Recorded work

Stage	Execution / output	Status
Specification review	Read six-page capstone source and mapped tasks to chosen healthcare use case	Complete
Corpus and docs	Created 100 FAQ records, printable PDF, project README, Windows guide, brain checkpoint and architecture source/image	Complete
RAG and agent workflow	Implemented FAISS + BM25-style rank fusion, LangChain retriever tool, LangGraph roles, session memory, validator/retry	Complete
API and UI	Implemented FastAPI health/query/feedback and Streamlit user interface	Locally smoke-tested
Local tests	Latest recorded run: 8 pytest tests passed; Ruff clean	Pass
Evaluation	Baseline vs. evidence-only agentic scores calculated for 20 FAQ prompts per mode	Complete; limitations noted
No-container CI/CD	Removed container artifacts; added Python CI and optional source-ZIP Azure App Service deployment	Workflow authored; no remote run evidence
Azure setup troubleshooting	User screenshot showed Project endpoint in legacy endpoint variable; user reported 503	Cause likely identified; no live Azure success captured

7.2 Azure configuration issue evidenced by screenshot

The supplied VS Code screenshot shows AZURE_OPENAI_ENDPOINT set to a URL of the form https://<resource>.services.ai.azure.com/api/projects/<project>. That is the Foundry Project endpoint, not the OpenAI-compatible model inference endpoint expected by the updated app. The user reported a 503 while running Streamlit against the older local setup. The visible API key in the supplied screenshot has been covered entirely in the report copy; the original image is not embedded.

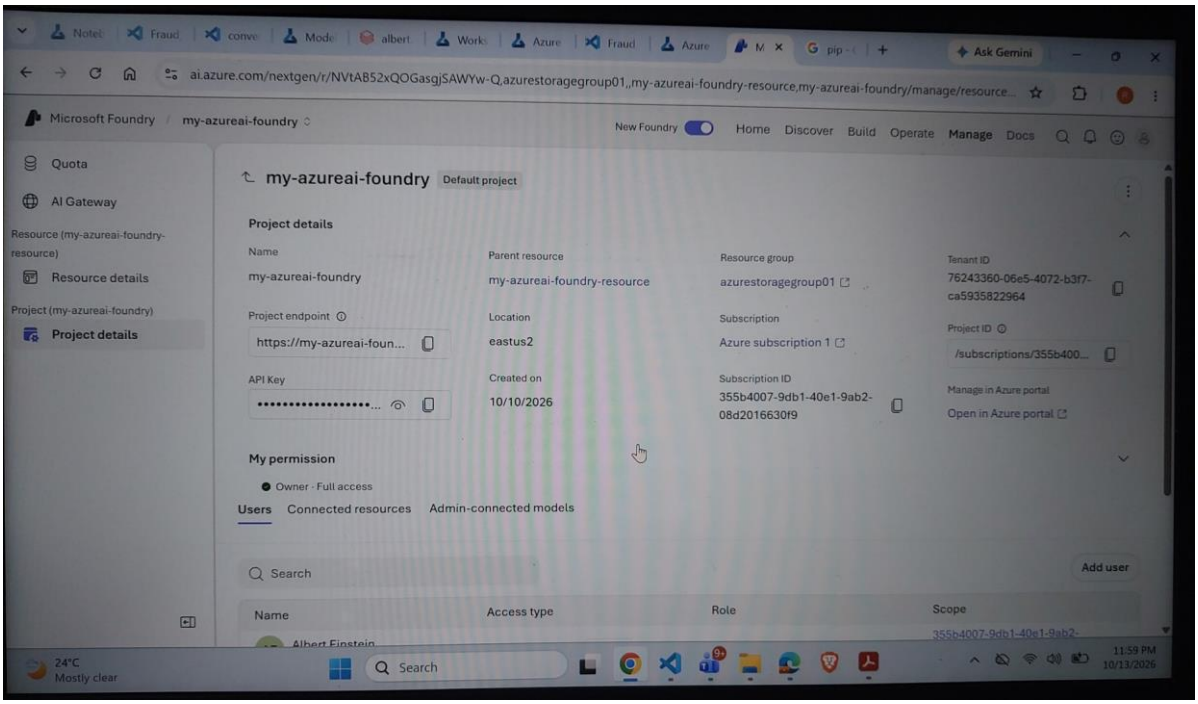


Figure 5. User-provided Foundry Project details. The portal's Project endpoint is distinct from the model inference endpoint.

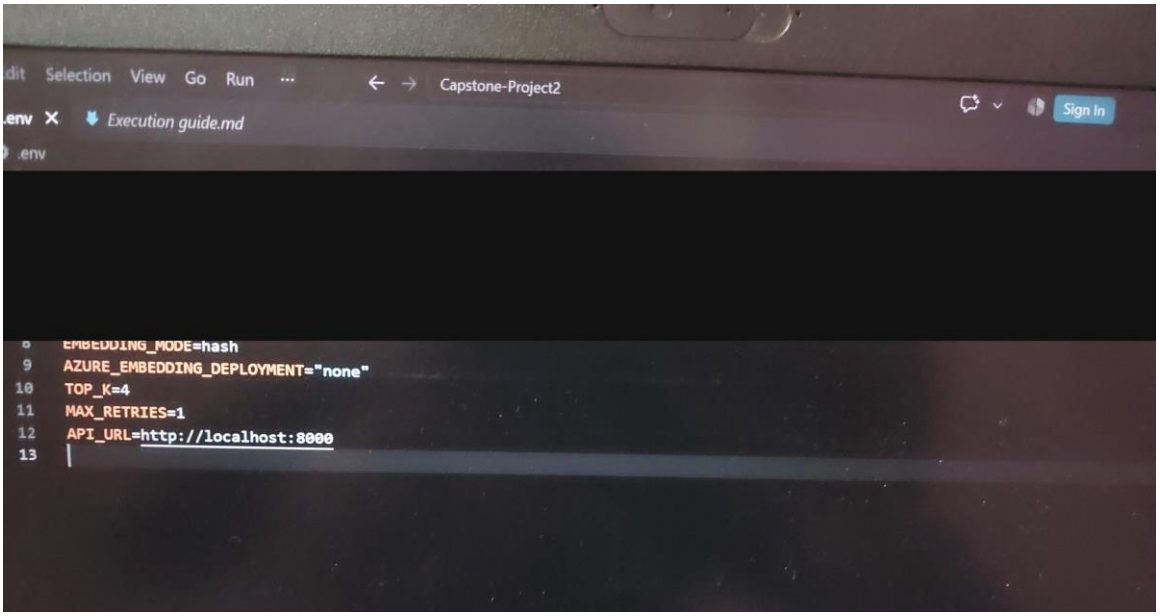


Figure 6. User-provided original .env screenshot. The complete Azure endpoint, credential and model configuration block is redacted in this report copy; the documented old Project-endpoint mismatch is described in the text.

Security note

The API key was legible in the submitted .env photograph. It has not been reproduced here. Rotate/revoke that key and use a replacement locally; do not attach .env files or unredacted screenshots to reports or repositories.

8. CI/CD, Deployment and Evidence Boundaries

8.1 Workflow design

The GitHub Actions workflow triggers on pushes and pull requests to main. The test job checks out the source, sets up Python 3.12, installs development requirements, runs Ruff and pytest, and on a main-branch push builds a source ZIP artifact containing the API source, application modules, data and runtime requirements. An optional deploy job uses an Azure App Service publish-profile secret and app-name variable, then can poll /health when DEPLOY_URL is configured. This design was adopted in lieu of container packaging at the user's request.

8.2 What is and is not verified

Item	Evidence status	Report interpretation
Local Ruff and pytest	Ruff clean; 8 tests pass	Verified in project environment
Local API health/query	HTTP service returned status ok; FAQ query produced sources and trace	Verified in evidence-only mode
Local Swagger/Streamlit	Real browser captures embedded above	Verified in local sandbox
GitHub Actions run	No GitHub run screenshot or run log supplied	Workflow is authored but remote CI is unverified
Azure live inference	No valid credential configured for sandbox execution; user reports prior 503	Not verified; do not claim GPT-5-mini call succeeded
Azure App Service deployment	No app name/publish profile or deployed URL evidence supplied	Not deployed/tested in this report

8.3 Azure configuration guidance

```
AZURE_OPENAI_BASE_URL=https://<resource>.services.ai.azure.com/openai/v1/
AZURE_OPENAI_API_KEY=<rotated resource inference key>
AZURE_OPENAI_DEPLOYMENT=gpt-5-mini
AZURE_OPENAI_MODEL=gpt-5-mini
```

The Project endpoint form ends in /api/projects/<project>; it must not be pasted as AZURE_OPENAI_BASE_URL for the direct inference-key pattern. Confirm the exact endpoint and key for the deployed model in Azure Foundry connection details. Save .env in the project root, stop/restart the API, and inspect GET /health for llm_configured and llm_configuration_issue. Never put the key in a report or source control. The latest code adds an actionable error for a project-endpoint mismatch and exposes API error detail in Streamlit.

This guidance describes the code's expected integration path. Because the sandbox run had no Azure credentials, these exact values and GPT-5-mini inference were not exercised as part of the recorded local test.

9. Conclusion, Limitations and Next Steps

9.1 Conclusion

The project delivers the core capstone architecture for healthcare FAQ retrieval and evidence-grounded response: a 100-record corpus, local FAISS hybrid retrieval, a reusable LangChain tool, a multi-step LangGraph, feedback endpoints, evaluation script, FastAPI, Streamlit, tests and a GitHub Actions deployment workflow. Local execution produced a real API response, citations and agent trace; the local user interface and Swagger API were captured for this report.

9.2 Limitations

- The latest measured generation evaluation ran with the no-Azure evidence-extractive fallback, not the deployed gpt-5-mini model.
- Deterministic feature-hash embeddings are not equivalent to trained semantic embeddings; hybrid keyword retrieval is lightweight and no reranker is present.
- The validation and confidence values are lexical heuristics, not calibrated confidence or clinical truth.
- Conversation memory and feedback logs are local/in-process; no patient data should be stored or entered.
- FAQ source links are broad public references; the corpus requires clinician review and jurisdiction-specific updates before any clinical use.
- No GitHub-hosted workflow execution, Azure model call, Azure endpoint health check or App Service deployment was captured as a successful run.

9.3 Recommended completion checklist

1. Revoke/rotate the API key visible in the original screenshot; save the replacement only in local .env or a secure secret store.

2. Use the Foundry model inference endpoint ending /openai/v1/ and the exact deployed model name; do not use the Project endpoint ending /api/projects/<project>.

3. Restart the FastAPI process after editing .env; verify /health and then POST /query; capture status and error detail with secrets redacted.

4. Push the repository to GitHub, configure Python CI and required Azure deployment secrets, and retain screenshots of a passing Actions run and deployed /health check.

5. If deploying for real healthcare use, obtain clinical review, apply privacy/security controls, and explicitly limit system scope to general education.

Appendix A. Local verification record

Representative commands used for the local smoke test and project checks. Run these from the repository root after activating the Python 3.12 virtual environment; the Windows setup guide contains the complete machine setup instructions.

```
ruff check src api ui tests
pytest -q
python -m src.evaluation.run_evaluation
uvicorn api.main:app --reload --host 127.0.0.1 --port 8000
streamlit run ui/main.py
```

Captured verification item	Recorded result
Automated tests	8 passed (latest recorded project test run)
Static lint	Ruff: all checks passed
Local API health	{"status": "ok", "llm_configured": false, "llm_configuration_issue": null, "faq_count": 100}
Local query trace	planner → retriever_agent → rag_tool → consultation_agent → grounding_validator
Azure/GitHub	Not exercised successfully in this report run; see evidence boundaries above

10. References

[1] Capstone Project 2: Multi-Agent RAG Pipeline with LangGraph, FastAPI and CI/CD Deployment. User-supplied six-page project brief.

[2] World Health Organization, Health topics — who.int/health-topics

[3] Centers for Disease Control and Prevention, Health topics — cdc.gov/health-topics.html

[4] LangGraph documentation, Quickstart — docs.langchain.com/oss/python/langgraph/quickstart

[5] LangChain documentation, FAISS vector store — docs.langchain.com/oss/python/integrations/vectorstores/faiss

[6] Microsoft Learn, Foundry model endpoints — learn.microsoft.com/azure/foundry/foundry-models/concepts/endpoints

[7] Microsoft Learn, LangChain with Foundry models — learn.microsoft.com/azure/foundry/how-to/develop/langchain-models

[8] Microsoft Learn, Foundry SDK and endpoint overview — learn.microsoft.com/azure/foundry/how-to/develop/sdk-overview

[9] Project source, FAQ corpus, evaluation output, architecture rendering and local API/UI execution evidence, 13–14 October 2026.